



**INDIAN INSTITUTE OF TECHNOLOGY
GUWAHATI**

Department of Electronics and Electrical Engineering

EE515 VLSI Architecture

Course Project Report

Implementation of the Sigmoid Block using CORDIC Architecture

Name	Yuvraj Singh
Roll Number	254102419
Course	EE515 VLSI Architecture
Project Title	Implementation of the Sigmoid Block using CORDIC Architecture
Language Used	Verilog HDL

Contents

1 Introduction	2
2 CORDIC Algorithm	2
3 Types of CORDIC	3
3.1 Circular CORDIC	3
3.2 Linear CORDIC	3
3.3 Hyperbolic CORDIC	3
4 Hyperbolic CORDIC Theory	4
5 Sigmoid using Hyperbolic CORDIC	4
6 Q2.14 Fixed Point Representation	5
7 Non-Restoring Divider	5
8 RTL Schematic	6
9 Simulation Results	6
10 Implementation Details	7
11 Advantages of the Design	7
12 Applications	8
13 Conclusion	8

1 Introduction

Artificial Neural Networks (ANNs) are widely used in machine learning accelerators and AI hardware systems. Activation functions are nonlinear mathematical functions used in neural networks to introduce nonlinearity into the learning process.

One of the most commonly used activation functions is the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (1)$$

The sigmoid function maps any real-valued input into the range from 0 to 1.

Direct hardware implementation of exponential functions is expensive because it requires:

- Multipliers
- Floating point arithmetic
- Large lookup tables
- Complex combinational logic

To reduce hardware complexity, the CORDIC algorithm is used. CORDIC computes transcendental functions using only:

- Shift operations
- Addition and subtraction
- Small lookup tables

Thus, CORDIC provides area-efficient and hardware-friendly implementation.

2 CORDIC Algorithm

CORDIC stands for Coordinate Rotation Digital Computer. It is an iterative algorithm used to compute:

- Trigonometric functions
- Hyperbolic functions
- Exponential functions
- Logarithmic functions
- Division and square root

The major advantage of CORDIC is that multiplication is replaced by shift-add operations.

The general iterative equations are:

$$x_{i+1} = x_i - d_i y_i 2^{-i} \quad (2)$$

$$y_{i+1} = y_i + d_i x_i 2^{-i} \quad (3)$$

$$z_{i+1} = z_i - d_i \vartheta_i \quad (4)$$

where:

- x_i and y_i represent vector coordinates
- z_i represents residual angle
- d_i determines rotation direction

3 Types of CORDIC

3.1 Circular CORDIC

Circular CORDIC is used for computing:

- Sine
- Cosine
- Tangent

The elementary angle is:

$$\vartheta_i = \tan^{-1}(2^{-i}) \quad (5)$$

3.2 Linear CORDIC

Linear CORDIC is used for multiplication and division operations.

3.3 Hyperbolic CORDIC

Hyperbolic CORDIC is used for computing:

- Hyperbolic sine
- Hyperbolic cosine

- Hyperbolic tangent
- Exponential functions

The elementary angle is:

$$\vartheta_i = \tanh^{-1}(2^{-i}) \quad (6)$$

This project uses Hyperbolic CORDIC because sigmoid can be expressed using hyperbolic tangent.

4 Hyperbolic CORDIC Theory

The hyperbolic rotation equations are:

$$x_{i+1} = x_i + d_i y_i 2^{-i} \quad (7)$$

$$y_{i+1} = y_i + d_i x_i 2^{-i} \quad (8)$$

$$z_{i+1} = z_i - d_i \tanh^{-1}(2^{-i}) \quad (9)$$

The initial conditions are:

$$x_0 = \frac{1}{K_h} \quad (10)$$

$$y_0 = 0 \quad (11)$$

$$z_0 = z \quad (12)$$

After multiple iterations:

$$x_n = \cosh(z) \quad (13)$$

$$y_n = \sinh(z) \quad (14)$$

The multiplication by 2^{-i} is implemented using arithmetic shifting.

5 Sigmoid using Hyperbolic CORDIC

The sigmoid function can be rewritten as:

$$\sigma(x) = \frac{1}{2} \left(1 + \tanh \frac{x}{2} \right) \quad (15)$$

Proof:

$$\tanh \frac{x}{2} = \frac{e^x - 1}{e^x + 1} \quad (16)$$

$$\Rightarrow \frac{1}{2} \ln \left(1 + \tanh \frac{x}{2} \right) = \frac{e^x}{e^x + 1} \quad (17)$$

$$= \frac{1}{1 + e^{-x}} \quad (18)$$

Thus sigmoid computation involves:

1. Computing $x/2$
2. Computing $\sinh(x/2)$ and $\cosh(x/2)$ using Hyperbolic CORDIC
3. Computing:

$$\tanh(x/2) = \frac{\sinh(x/2)}{\cosh(x/2)} \quad (19)$$

4. Generating final sigmoid output

6 Q2.14 Fixed Point Representation

The project uses 16-bit signed fixed-point representation in Q2.14 format.

Sign Bit	Integer Bits	Fractional Bits
1	1	14

Scaling factor:

$$2^{14} = 16384 \quad (20)$$

Examples:

Decimal Value	Q2.14 Integer
1.0	16384
0.5	8192
0.25	4096
-1.0	-16384

7 Non-Restoring Divider

To compute:

$$\tanh(x/2) = \frac{\sinh(x/2)}{\cosh(x/2)} \quad (21)$$

A non-restoring divider is used.

The divider works iteratively:

1. Shift remainder left
2. Compare with divisor
3. Perform subtraction or addition
4. Generate quotient bit

Advantages:

- Area efficient
- Multiplier-less implementation
- Sequential hardware realization
- Low complexity

8 RTL Schematic

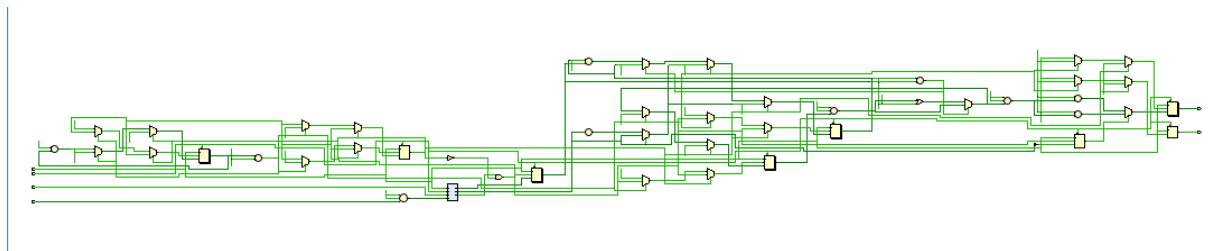


Figure 1: RTL Schematic of the Implemented Design

9 Simulation Results

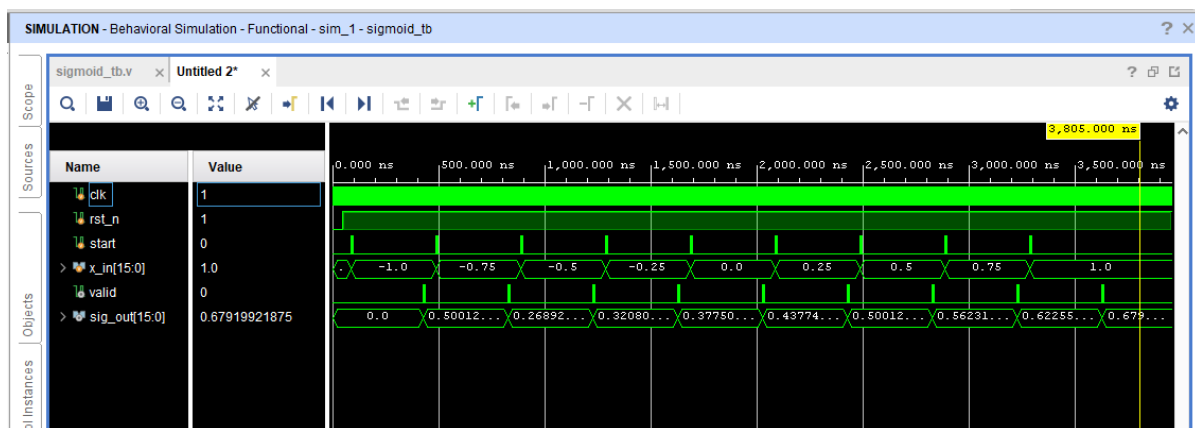


Figure 2: Simulation Results of Sigmoid Block

The waveform verifies correct sigmoid computation.

Input	Sigmoid Output
-1.0	0.2689
-0.5	0.3775
0.0	0.5001
0.5	0.6225
1.0	0.6792

10 Implementation Details

The Verilog implementation contains:

- Hyperbolic CORDIC Core
- Sigmoid Module
- Non-Restoring Divider
- Testbench

Main design features:

- 16-bit datapath
- Q2.14 representation
- Shift-add arithmetic
- Lookup-table based angles
- Sequential iterative computation

11 Advantages of the Design

The proposed architecture provides:

- Multiplier-less implementation
- Low hardware area
- Low power consumption
- FPGA and ASIC compatibility
- Good computational accuracy

12 Applications

Applications include:

- Neural network accelerators
- AI hardware
- FPGA based machine learning systems
- Embedded AI inference engines

13 Conclusion

A hardware-efficient sigmoid function was implemented using Hyperbolic CORDIC architecture in Verilog HDL.

The architecture uses:

- Shift-add operations
- Fixed-point arithmetic
- Hyperbolic vector rotations
- Non-restoring division

The design avoids expensive floating-point arithmetic and hardware multipliers while maintaining good numerical accuracy.

Simulation results verify the correctness of the implementation.

The proposed design is highly suitable for low-area and low-power neural network accelerators.